PAPER TRAIL

An Agentic, Evidence-Grounded Fake News Detection Platform

Capstone Project Report: Architecture, Implementation, Workflow and Technologies

Team member	Component owned
Kavish Basole	ML models, agentic verification pipeline, website backend API
Phani Nishik Kalidindi	Feature engineering and website frontend
Sashank Solasa	Origin / propagation tracing and mobile frontend
Virat Tallapally	Source credibility, analytics and mobile backend

Registration numbers: _____ Guide / Faculty: _____

Report date: 30 September 2026

Rubric map. Originality & progress: Sections 2, 11, 12 · Method: Sections 4, 5, 8, 9 · Implementation: Sections 6, 7, 10 · Presentation: Sections 3, 13, Appendices

Contents

1. Abstract

2. Introduction, problem statement and originality

3. System architecture

4. Methodology: research approach and design decisions

5. Data and preprocessing

6. Machine-learning models (implementation)

7. The agentic verification pipeline (implementation)

8. Fusion, calibration and abstention

9. Evaluation methodology and results

10. Backend API, database, security and frontend

11. Testing and engineering quality

12. Project progress, limitations and future work

13. Technology stack summary

14. Running the system

15. Conclusion

Appendix A. API reference · B. Configuration · C. Repository map · D. Glossary

1. Abstract

Paper Trail is a fake-news detection platform that goes beyond classifying writing style. Conventional classifiers learn *how* fake news is written, and they are easily fooled by a well-written fabrication. Paper Trail combines two independent signals. The first is a **stylistic ensemble** of 22 machine-learning models combined by a stacking meta-learner. The second is an **evidence channel**: an agentic LangGraph pipeline extracts check-worthy claims from an article, retrieves external evidence (Wikipedia and NewsAPI), and checks each claim against that evidence with a DeBERTa-v3 natural-language-inference (NLI) model.

A calibrated logistic-regression **adjudicator** fuses 13 features from both channels into one probability. A two-threshold abstention band sends uncertain articles to a **human review queue** instead of guessing. The system accepts plain text (English or Hindi) and images (screenshots, via OCR). It is served by a FastAPI backend with JWT authentication and a Next.js web application.

Measured results: the stylistic ensemble reaches 92.1% accuracy on an in-distribution hold-out. The fused system commits verdicts with **98.3% accuracy at 42% coverage** on the hold-out slice and **88.2% accuracy at 57% coverage** on novel, hand-written articles, with zero false "fake" verdicts on either slice. The report also states the system's limitations openly (Section 12).

2. Introduction, Problem Statement and Originality

2.1 Problem statement

Misinformation spreads faster than manual fact-checkers can respond. Automated detectors exist, but most share three weaknesses: (a) they classify *style*, not truth, so a polished fabrication passes; (b) they always answer, even when they should not, so they give confident wrong verdicts; and (c) they offer no explanation, so users have no reason to trust the output. India adds two further problems: much misinformation arrives as **images** (WhatsApp forwards, screenshots) and in **Hindi**.

2.2 Objectives

- Build and benchmark a broad bank of classical, deep-learning and hybrid text classifiers on a large merged corpus.
- Build an agentic pipeline that verifies the *claims* in an article against external evidence, with a traceable reasoning record.
- Fuse the stylistic and evidence signals into one calibrated score and **abstain** when confidence is low.
- Support image input (OCR) and Hindi input (translation) as entry points into the same pipeline.
- Expose everything through a documented REST API, a human review workflow and a polished web interface.
- Evaluate honestly on leak-free splits and on novel articles the models have never seen.

2.3 What is original in this project

Contribution	Why it is original / non-trivial
Two-channel verification (style + evidence)	Most student projects stop at a TF-IDF classifier. Here the style score is one input to a fusion model, and claim-level evidence carries the truth signal.
Claim-level NLI with trust-weighted aggregation	Each claim is scored as entailed, contradicted or neutral. Weights combine relevance and source trust, and contradiction is held to a stricter bar than entailment.

Contribution	Why it is original / non-trivial
"Absence of evidence is not contradiction"	Retrieval failures (no hits, rate limit, error) are typed and never counted as contradicting evidence. This is a common bug in naive fact-checkers.
Calibrated abstention with a precision target	Thresholds are chosen on dev data to hit 90% worst-class precision, then low-confidence cases are routed to human review.
Per-stage fault isolation (graceful degradation)	Each node runs under a timeout and a typed fallback, and records a *Degradation* instead of crashing.
Multimodal and multilingual entry	Image → OCR (Hindi + English) and Hindi → English translation feed the same verifier.
Leak-free evaluation harness	Fixed-seed splits, a hand-written golden set, degradation tests and a documented audit of known evaluation artefacts (dateline leak, stacking leakage, adversarial memorisation).
Deterministic, offline test suite	123 tests run with no network: retrieval monkeypatched, NLI stubbed, pipeline mocked.

3. System Architecture

The system has four layers. The Next.js frontend talks to the FastAPI backend over JSON. The backend runs the agentic core (`paper_trail_core`) and persists results in a relational database. Trained model artefacts are loaded once at start-up through a model registry.

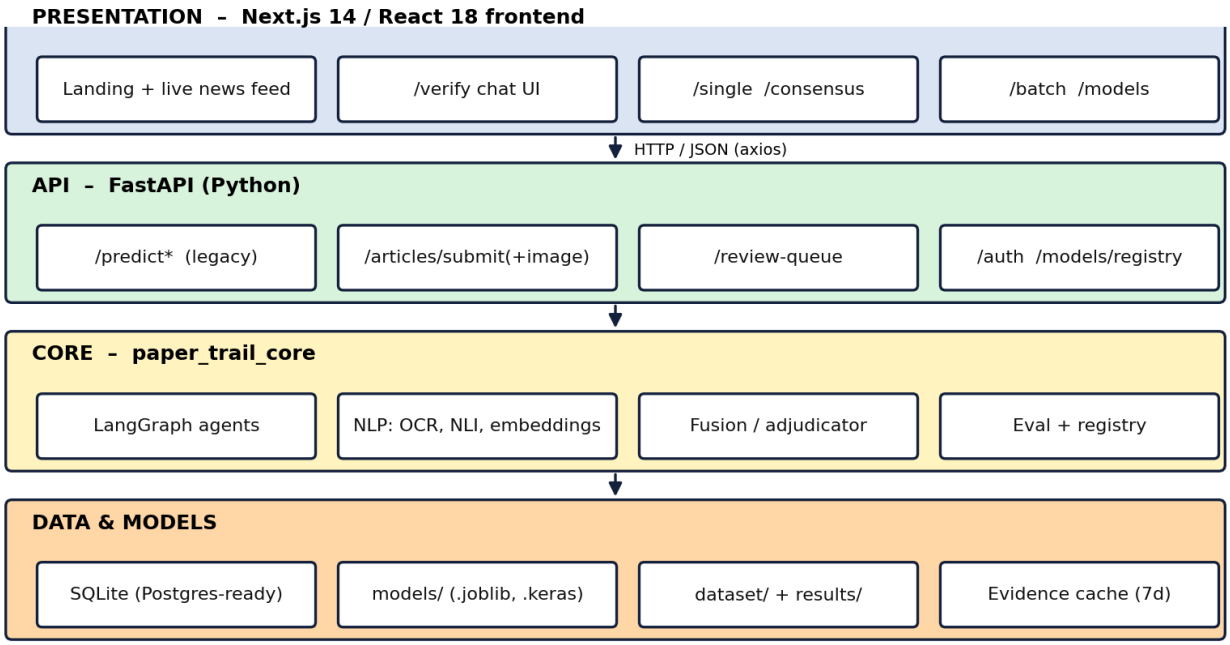


Figure 1. Layered architecture of Paper Trail.

3.1 Component responsibilities

Layer / module	Responsibility
frontend/ (Next.js 14)	Landing page with live news feed; /verify chat UI for the agentic pipeline; /single, /consensus, /batch and /models pages for the classifier bank.
api/ (FastAPI)	REST endpoints, request validation (Pydantic), JWT auth, background jobs, SQLAlchemy ORM, model loading.
paper_trail_core/agents/	LangGraph nodes: OCR, translation, claim extraction, evidence retrieval, verification, stylistic ensemble, fusion and adjudication.

Layer / module	Responsibility
paper_trail_core/nlp/	OCR, language ID, NLI, sentence embeddings, sentence segmentation, check-worthiness scoring, numeric-conflict detection.
paper_trail_core/eval/	Leak-free splits and metrics (accuracy, F1, ROC-AUC, ECE, coverage, precision).
paper_trail_core/registry/	Model registry store and CLI (`list`, `top`).
models/	Trained artefacts: 18 classical (.joblib, about 1.1 GB), 4 deep-learning (.keras, about 193 MB), stacking meta-learner, fusion adjudicator, registry.json.
notebooks/, scripts/, results/	24 training/analysis notebooks; evaluation, fusion-training and registry scripts; result CSV/JSON/Markdown.

3.2 Two systems in one application

The repository contains (1) a **legacy classifier bank**, served by unauthenticated `/predict*` endpoints so that users can compare individual models, and (2) the **agentic verification pipeline**, which is the capstone deliverable, served by `/articles/submit`. Both share the model registry and the same database and API process.

4. Methodology: Research Approach and Design Decisions

4.1 Overall approach

The work followed an iterative, evidence-driven method. (1) Build a strong baseline bank and benchmark it fairly. (2) Audit the baseline and find where it fails, in particular on novel articles. (3) Add an evidence channel to address those failures. (4) Fuse the channels and calibrate. (5) Evaluate on leak-free and novel data, and document the limits. Each step is backed by committed code, notebooks and result files.

4.2 Key design principles

- **Verify claims, not style.** The style model cannot know whether a claim is true, so external evidence decides.
- **Never guess.** A third verdict, *uncertain*, plus a review queue is better than a confident wrong answer.
- **Typed failure, not silent failure.** Every stage records why it degraded. Raw exception text is never stored or returned.
- **Read labels from config, never assume.** NLI label order comes from the model's `id2label`, which avoids a classic label-swap bug.
- **Local-first.** The default backends (heuristic claim extraction, Wikipedia retrieval, local NLI) need no paid API key.
- **Reproducibility.** Fixed seeds (`random_state=42`, langdetect seed 0), frozen feature order asserted on load, and a registry of every model.

4.3 Design decisions and alternatives

Decision	Chosen	Alternative considered	Reason
Claim extraction	Heuristic regex segmenter + lexical check-worthiness	LLM extraction (implemented, optional)	Free, deterministic, offline. The LLM backend carries a grounding guard.

Decision	Chosen	Alternative considered	Reason
NLI model	MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli	Fine-tuned XLM-R (planned Model 3)	Strong off-the-shelf, trained on MNLI + FEVER + ANLI; about 370 MB runs on CPU.
Retrieval	Wikipedia (keyless) + NewsAPI	Tavily, SerpAPI (adapters exist)	Works with zero keys; other providers are pluggable.
Fusion	Calibrated logistic regression	Gradient boosting / neural fusion	Small dev set (102), so an interpretable low-variance model with readable coefficients suits it.
Translation	LLM (Claude Haiku 4.5 / gpt-4o-mini)	IndicTrans2	Faster to integrate; preserves named entities; prompt-injection-aware.
OCR	Tesseract (hin+eng)	PaddleOCR	Mature, offline, Hindi language pack available.
Orchestration	LangGraph StateGraph	Plain function chain	Conditional edges (image vs text, abstain) and a clear, inspectable graph.
Compute	CPU locally; GPU training on Colab	Local RTX 4050 (6 GB)	CUDA torch install failed on low free memory, so heavy training moves to Colab.

5. Data and Preprocessing

5.1 Datasets

Five public datasets are merged into one corpus of **115,088 records** (Fake 49,486, Real 65,602). After cleaning and de-duplication **104,521** remain. A synthetic adversarial set is added, oversampled 15× in the training split.

Source	Raw records	After dedup	Notes
ISOT	44,898 (23,481 fake / 21,417 real)	38,826	News articles; contains a Reuters-dateline artefact (see 12.2)
FakeNewsNet	23,196	21,046	PolitiFact and GossipCop
LIAR	12,791	12,726	Short political statements
MultiFC	31,921	31,919	Multi-domain fact-check claims
BuzzFeed	2,282	4	Almost entirely duplicates
Adversarial (synthetic)	n/a	n/a	Generated by script; 15× oversampled in training
Golden set (hand-written)	30 (16 real / 14 fake)	n/a	Evaluation only, never trained on

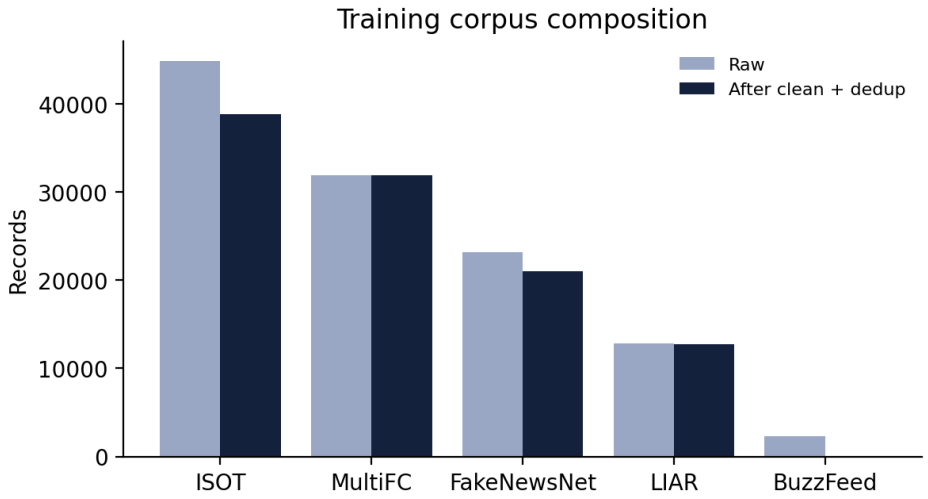


Figure 2. Corpus composition before and after cleaning and de-duplication.

5.2 Preprocessing

- Text cleaning: lowercase, strip URLs and non-letter characters (`preprocess.py`).
- Split: stratified 80/20, `random_state=42`. Train 83,616 (34,126 fake / 49,490 real); test 20,905 (8,532 fake / 12,373 real). Class balance 40.8% fake, 59.2% real.
- Sparse features: TF-IDF with 50,000 features and unigrams + bigrams.
- Dense features: TF-IDF (5,000) followed by TruncatedSVD (200 components) for models that need dense input.
- Deep-learning features: Keras tokenizer, vocabulary 30,000, sequence length 200.

6. Machine-Learning Models (Implementation)

All models are trained by `export_models.py` (flags `--skip-dl`, `--skip-classical`, `--skip-hybrid`) and analysed in 24 notebooks that produce ROC, PR, confusion-matrix, learning-curve and feature-importance figures.

6.1 Classical models (18)

Logistic Regression; Multinomial, Bernoulli and Complement Naive Bayes; Linear SVM; Decision Tree (depth 50); Random Forest (100 trees); Extra Trees; AdaBoost; Ridge; SGD (modified Huber); Passive-Aggressive; Perceptron; MLP; KNN; Gradient Boosting; XGBoost; LightGBM; CatBoost.

6.2 Deep-learning models (4, Keras / TensorFlow)

LSTM (128 units), GRU (128), CNN (Conv1D 128 filters, kernel 5) and CNN+LSTM (Conv1D 64 → LSTM 64). Embedding dimension 128, 10 epochs, batch size 64.

6.3 Hybrid stacking ensemble

A logistic-regression meta-learner is trained on the base models' predicted probabilities (`models/hybrid/stacking_lr.joblib`, 22 base models). It is the "stylistic ensemble" that feeds the pipeline's `ensemble_score` (its probability of *real*).

6.4 Benchmark results

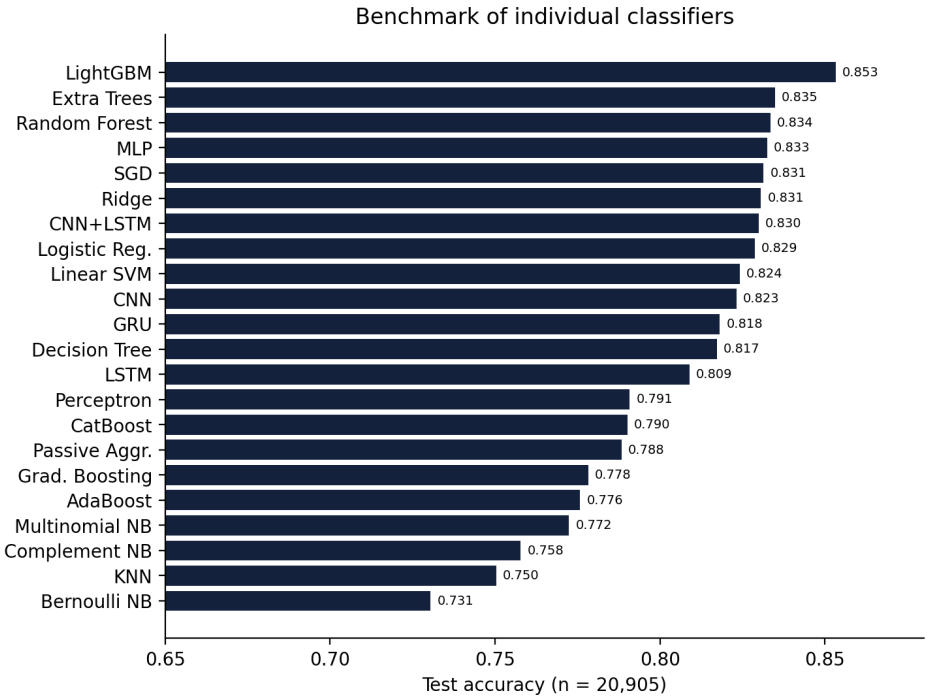


Figure 3. Test accuracy of the individual classifiers on 20,905 held-out articles.

Model	Accuracy	F1	ROC-AUC
LightGBM (best)	0.8532	0.8501	0.9382
Extra Trees	0.8349	0.8318	0.9092
Random Forest	0.8335	0.8299	0.9109
MLP	0.8325	0.8297	0.9271
CNN+LSTM (best deep)	0.8299	0.8282	0.9228
Logistic Regression	0.8287	0.8267	0.9183
CNN	0.8232	0.8237	0.9199
GRU	0.8181	0.8136	0.9172
LSTM	0.8089	0.8054	0.9028
Bernoulli NB (lowest)	0.7305	0.7203	0.7245

Training times range from 0.02 s (KNN) to 2,125 s (LSTM). Benchmark figures cover 22 models in `results/all_results.csv`; the full table is reproducible with notebook 23.

6.5 Planned models (Models 1, 2, 3)

The architecture document specifies three further models for multilingual and regional verification, trained on Colab GPU. **These are planned, not built**, and no results are claimed for them.

Model	Plan
Model 1	Fine-tune MuRIL / IndicBERT as a multilingual stylistic classifier.
Model 2	Region classifier (DistilmBERT / MuRIL-base) with 6 coarse classes: National, International, North, South, East, West.
Model 3	Fine-tune XLM-R / mDeBERTa for claim NLI, baselined against the current DeBERTa-v3.

7. The Agentic Verification Pipeline (Implementation)

The pipeline is a LangGraph `StateGraph` in `paper_trail_core/agents/pipeline.py`. Entry is conditional on input type. Public functions are `run_verification_pipeline(text, url)` and `run_verification_pipeline_image(path, url)`.

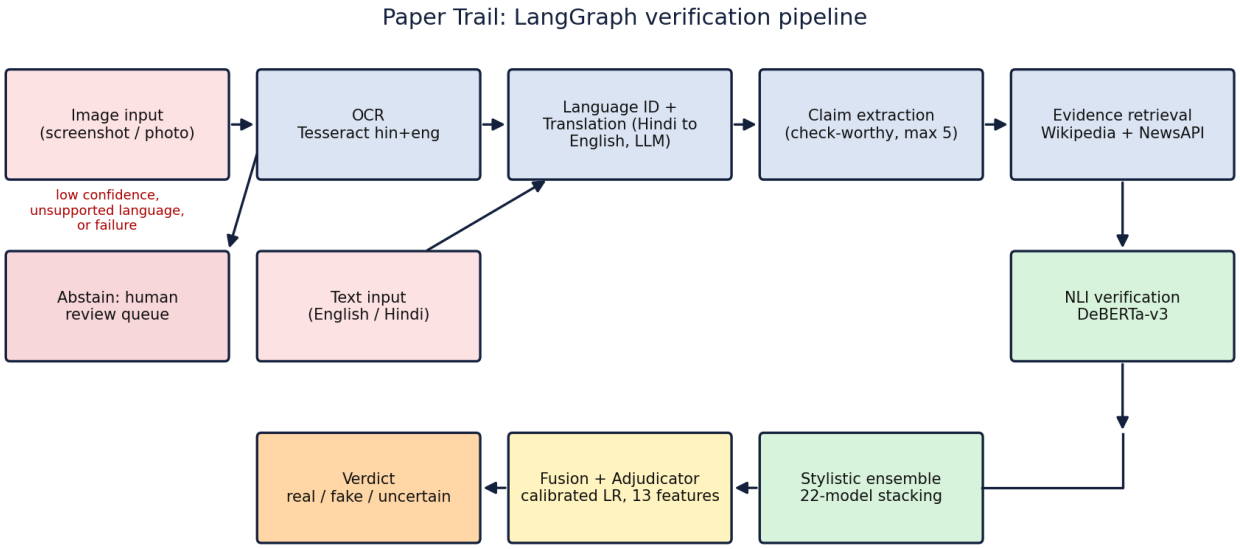


Figure 4. Workflow of the verification pipeline, including the abstain path.

7.1 Fault isolation

Every node runs inside a `ThreadPoolExecutor` with a timeout (`STAGE_TIMEOUT_S` = 90 s) and a typed fallback. A timeout or exception records `Degradation(stage, kind, detail, severity)`. Input length is bounded to 40-50,000 characters.

7.2 Stage by stage

Stage 1: OCR (images only)

Tesseract via pytesseract with `hin+eng`. Preprocessing with PIL: EXIF transpose, grayscale, autocontrast, upscale to at least 1,000 px. Confidence is the mean per-word confidence; below 0.55 the pipeline abstains. Script is tagged Devanagari or Latin by regex. Backends: *tesseract*, *mock*, *off*.

Stage 2: Language ID and translation

`langdetect` (seed 0) supports English and Hindi. English passes through unchanged. Hindi is translated to English with an LLM (Claude Haiku 4.5 by default; gpt-4o-mini alternative) using a JSON-only, prompt-injection-aware prompt, capped at 8,000 characters (truncation flagged). The original text is preserved. Other languages abstain.

Stage 3: Claim extraction

A regex sentence segmenter protects decimals, abbreviations and initials. A lexical **check-worthiness** score keeps claims above 0.35. Near-duplicates are removed with Jaccard similarity and then embedding cosine 0.92. At most 5 claims are kept, in document order.

Stage 4: Evidence retrieval

- Query built from named entities plus high-IDF words (IDF from the trained TF-IDF).

- Sources: Wikipedia (keyless) and NewsAPI with a trusted-domain filter; Tavily and SerpAPI adapters are available.
- **Relevance gate:** snippets are embedded with `all-MiniLM-L6-v2` and discarded below cosine 0.45.
- **Source trust tiers:** wire services 0.95-1.0, major outlets 0.8-0.9, reference sites (who.int, nasa.gov) 0.85-0.95, unknown 0.5.
- Disk cache (7-day TTL), 3 parallel workers, at most 12 calls, 30-minute rate-limit cooldown.
- Typed statuses: *no_hits, no_relevant, rate_limited, error*. None of these counts as contradiction.

Stage 5: NLI verification

Model: **MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli**. Each evidence snippet is split into sentences and each sentence is a premise for the claim (hypothesis). Inference is batched (32), fp16 forward with fp32 softmax. Per-claim aggregation:

```
w = scaled_relevance x trust
entail_mass = max( w * P(entailment) )
contra_mass = max( w * P(contradiction) )
support     = entail_mass - effective_contradiction    in [-1, 1]
```

Neutral contributes nothing. A **numeric-conflict detector** compares percentages in points, years categorically and other figures relatively. Claim verdicts: *entailment, contradiction, neutral, unverifiable*. If torch is unavailable a lexical fallback runs and forces abstention.

Stage 6: Stylistic ensemble

Calls the registry's `hybrid_stacking` model and takes `prob_real`. Fallback is 0.5 with a recorded degradation.

Stage 7: Fusion and adjudication

Covered in Section 8.

Abstain node

Terminal node returning *uncertain*, confidence 0 and `needs_human_review = True`. Reached after failed or low-confidence OCR, unsupported language, or failed translation.

7.3 Output contract

`VerdictResult` carries: verdict, confidence, per-claim verdicts (with evidence snippets, support, entail/contradict mass, relevance, numeric conflict, sources, status), ensemble score, fused score, fusion method and features, reasoning trace, human-review flag, degradations and retrieval statistics. The API persists this and the frontend renders it as verdict card, claim list, evidence trail and trace.

8. Fusion, Calibration and Abstention

The 13-feature vector (`FEATURE_NAMES`, frozen and asserted on load):

```
ensemble_score, mean_support, max_support, min_support, entail_mass_max,
contradict_mass_max, coverage, informative_ratio, n_claims_norm, mean_relevance,
mean_trust, numeric_conflict_max, nli_degraded
```

- **Model:** logistic regression with sigmoid calibration (`models/fusion/adjudicator_v1.joblib`), trained by `scripts/train_fusion.py`.
- **Thresholds:** *likely_real* if P ≥ 0.605; *likely_fake* if P ≤ 0.395; otherwise *uncertain*. Chosen so worst-class precision meets the 0.90 target (met), expected coverage 96.1% on dev.
- **Dev metrics** (n = 102): accuracy 0.8922, macro-F1 0.8921, ROC-AUC 0.9438.

- **Most influential coefficients:** ensemble_score +2.06, coverage +0.53, n_claims_norm +0.51, informative_ratio -0.37, mean_trust -0.37.
- **Hard gates:** abstain if NLI ran degraded or no claim could be checked.
- **Fallback:** with no trained model, a heuristic blend runs with the evidence weight capped at 0.85.
- Abstained articles are flagged and inserted into the `review_queue` table for a human reviewer.

9. Evaluation Methodology and Results

9.1 Method

- **Holdout slice:** 140 articles (70 per class) from the leak-free 20% partition.
- **Golden slice:** 30 novel hand-written articles the models never saw (16 real, 14 fake).
- **Channels compared:** ensemble-only, evidence-only, fused.
- **Metrics:** accuracy, macro-F1, ROC-AUC, expected calibration error (ECE), committed coverage and committed precision, and latency.
- Scripts: `scripts/evaluate_pipeline.py`, outputs in `results/eval_*.md/json/csv`.

9.2 Results

Slice	Channel	Accuracy	Macro-F1	ROC-AUC	ECE
Golden (n=30)	ensemble-only	0.600	0.489	0.846	0.341
Golden (n=30)	evidence-only	0.633	0.597	0.819	0.102
Golden (n=30)	**fused**	**0.667**	**0.625**	**0.915**	0.235
Holdout (n=140)	ensemble-only	0.921	0.921	0.983	0.045
Holdout (n=140)	evidence-only	0.421	0.361	0.459	0.033
Holdout (n=140)	**fused**	**0.929**	**0.929**	0.966	0.154

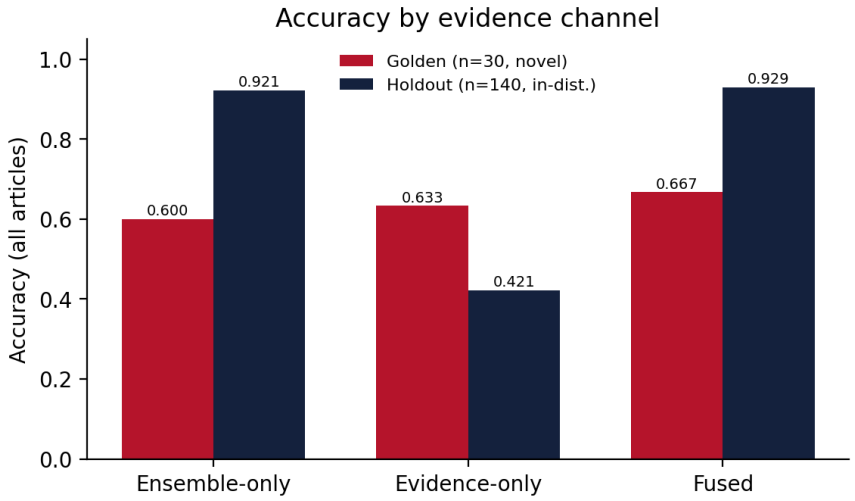


Figure 5. Accuracy over all articles, by channel and evaluation slice.

9.3 Committed (non-abstained) verdicts

Slice	Coverage	Accuracy on committed	Precision (fake / real)	Confusion
Golden	56.7% (13 of 30 sent to review)	0.882	1.000 / 0.867	TP 13, TN 2, FP 2, FN 0
Holdout	42.1% (81 of 140 sent to review)	0.983	1.000 / 0.972	TP 35, TN 23, FP 1, FN 0

Headline: 98.3% accuracy in-distribution (42% coverage) and 88.2% on novel articles (57% coverage), with zero false "fake" verdicts on either slice. On golden, the abstention band 0.27-0.73 met the 0.90 target precision (worst-class 0.917 at 43.3% coverage). Median latency is about 0.8-1.0 s (p95 1.3-2.1 s).

9.4 Interpretation

- **Why fusion matters.** On novel articles the style ensemble alone scores 0.600, and fusing evidence lifts accuracy to 0.667 and ROC-AUC from 0.846 to 0.915. Example: a fabricated "Stanford superconductor" story scored 0.926 real by the ensemble and was called fake (fused 0.341).
- **Why the evidence channel looks weak on holdout.** Holdout articles are dated 2015-2017, outside NewsAPI's roughly one-month window, so only 17-18% of claims found evidence. The channel is retrieval-starved there, and the model correctly abstains.
- **Per-category weakness on golden:** conspiracy spam 0.50, plausible fabrication 0.33, health misinformation 0.33, financial scam 0.00 (small n; flagged as future work).

10. Backend API, Database, Security and Frontend

10.1 FastAPI backend (api/)

FastAPI 2.0.0 with a lifespan hook that creates tables, runs lightweight column migrations, loads the model registry and optionally warms the NLI and embedding models (`WARM_MODELS`). Swagger UI is served at `/docs`. Input is validated with Pydantic (text 5-50,000 characters).

Endpoint	Auth	Purpose
GET /, /health	no	Service and model-load status
GET /models	no	List loaded classifiers
POST /predict	no	Single model prediction (text, model_name, threshold)
POST /predict/all	no	All models + majority-vote consensus and confidence
POST /predict/batch	no	Up to 100 texts (413 beyond)
GET /results, /results/charts[/{name}]	no	Benchmark table, best model, chart images
POST /auth/token	-	JWT (HS256, 60 min)
POST /articles/submit	JWT	Submit text; returns 202 and runs pipeline in a background task
POST /articles/submit-image	JWT	Multipart image (jpeg/png/webp/bmp/tiff, ≤10 MB); returns 202
GET /articles/{id}	no	Poll status, stage and verdict
GET /review-queue	no	Paginated queue, filter by status
POST /review-queue/{id}/resolve	JWT	Human verdict: likely_fake / likely_real / uncertain + notes
GET /models/registry	no	Filter and sort the model registry

10.2 Asynchronous workflow

Submission returns 202 immediately with an article id. A FastAPI background task runs the pipeline in its own DB session and updates the article's *stage* as it progresses. The client polls `GET /articles/{id}` (1.5 s interval, 180 s timeout) until *completed* or *failed*. This keeps the UI responsive during NLI and retrieval.

10.3 Database (SQLAlchemy 2, SQLite, Postgres-ready)

Table	Key columns
articles	id, url, text, status, stage, submitted_at
verdicts	id, article_id, verdict, confidence, claim_verdicts (JSON), ensemble_score, fused_score, reasoning_trace, needs_human_review, degradations (JSON), ai_model_version, created_at
review_queue	id, article_id, verdict_id, status, assigned_to, created_at, resolved_at, human_verdict, human_notes

10.4 Security

- JWT (HS256, python-jose), 60-minute expiry; passlib/bcrypt available.
- Uploads: content-type allow-list, 10 MB limit, random temp filenames.
- LLM prompts are written to resist prompt injection; raw exception text is never returned.
- **Demo-grade caveats (stated openly):** two hard-coded demo users, CORS open to all origins, and `GET /articles/{id}` unauthenticated. These must be replaced (user table, restricted origins, secret from environment) before production.

10.5 Frontend (Next.js 14)

App Router, React 18, plain CSS (no Tailwind), axios, framer-motion. Dark theme by default with persisted light toggle. Brand: "The Daily Truth" site with the "Paper Trail" verification page.

Route	Function
/	Landing page, live headlines (RSS from Times of India and Hindustan Times via /api/news, no key), feature cards
/verify	Chat-style verification: type text or upload / paste an image. Shows verdict card, per-claim status (SUPPORTED / CONTRADICTED / NO CLEAR SIGNAL / COULD NOT VERIFY), evidence trail, reasoning trace and degradations
/single	Single-article prediction with model selector, confidence gauge and probability bar
/consensus	"Editorial Consensus": all models vote
/batch	Batch analysis of multiple articles
/models	Model leaderboard

Reusable components: Navbar, Logo, ThemeToggle, Footer, NewsFeed, FeatureCards, ChatComposer, VerdictCard, ConfidenceGauge, ProbabilityBar, ModelSelector, EmptyState, ErrorState, ErrorBoundary, LoadingOverlay. `services/api.js` centralises all calls (base URL from `NEXT_PUBLIC_API_URL`, 30 s timeout).

11. Testing and Engineering Quality

The pytest suite contains **123 test functions** and runs fully offline (retrieval monkeypatched, NLI stubbed, pipeline mocked).

Test file	Tests	Focus
test_adjudicator.py	16	Fusion, thresholds, hard gates, fallback
test_api.py	17	Endpoints, auth, validation, review queue
test_claim_extractor.py	12	Segmentation, dedup, decimals not splitting claims
test_ensemble.py	6	Stylistic ensemble and fallback
test_evidence_retriever.py	23	Queries, trust tiers, cache, typed statuses
test_ocr_agent.py	10	Confidence gate, script tagging, abstention
test_pipeline_e2e.py	10	End-to-end and the JSON contract the frontend reads
test_translation_agent.py	12	Pass-through, Hindi, truncation, injection
test_verifier.py	17	NLI labels from config, numeric conflicts, aggregation

Regression tests pin past bugs: decimals must not split claims, NLI labels are read from config, and absence of evidence never yields contradiction. Configuration is typed via pydantic-settings with per-stage backend flags (`CLAIM_BACKEND`, `NLI_BACKEND`, `EVIDENCE_BACKEND`, `ENSEMBLE_BACKEND`, `OCR_BACKEND`, `TRANSLATION_BACKEND`).

12. Project Progress, Limitations and Future Work

12.1 Progress summary

Area	Status
Dataset merge, cleaning, leak-free splits	Done
22-model classifier bank + notebooks	Done
Stacking ensemble	Done (leakage fix pending)
Agentic pipeline: OCR, translation, claims, retrieval, NLI, fusion, abstention	Done
FastAPI backend, JWT, DB, review queue, model registry	Done
Next.js frontend with image upload and redesign	Done (recent changes not yet committed)
Evaluation harness, golden set, fusion training	Done
Tests (123)	Done
Models 1/2/3, region classifier, FAISS regional retrieval, rationale agent, LLM gateway, 100+ case golden set	Planned (Colab GPU)
Teammate modules: propagation tracing, source credibility, mobile app	Owned by teammates (not in this repository)

12.2 Known limitations (documented honestly)

- The stacking ensemble classifies **style, not truth**. On novel text it scored conspiracy spam 0.695 real and a fabricated Federal Reserve story 0.961 real. This is exactly why the evidence channel exists.
- **Stacking meta-learner leakage:** meta-training rows are a subset of the base models' training data. The fix is out-of-fold predictions (not yet done).
- **ISOT dateline artefact:** real ISOT articles start with a Reuters dateline, which inflates ISOT accuracy to 1.000.
- **Adversarial set** returns 300/300 because of 15× oversampling. It is a memorisation baseline, not a robustness claim.

- Retrieval is English-only with a tight NewsAPI quota (100 requests/day, about 1-month window).
- The golden set is small (n = 30), so its confidence intervals are wide.
- No evaluation yet of OCR accuracy, Hindi translation quality or multilingual / regional performance.
- No Docker or CI configuration yet.

12.3 Future work

- Train Models 1-3 on Colab and benchmark against the current off-the-shelf DeBERTa-v3.
- Region-partitioned FAISS retrieval cascade (LOCAL → GLOBAL → MANUAL REVIEW) driven by curated source lists.
- LLM gateway with guardrails and a rationale agent for natural-language explanations.
- Out-of-fold stacking; expand the golden set to 100+ cases; add Telugu and Tamil.
- Containerise (Docker), add CI, and replace demo auth with a real user store.

13. Technology Stack Summary

Area	Technologies
Language / runtime	Python 3, JavaScript (ES modules), Node.js
Classical ML	scikit-learn, XGBoost, LightGBM, CatBoost, scipy, pandas, numpy, joblib
Deep learning	TensorFlow / Keras (LSTM, GRU, CNN, CNN+LSTM); PyTorch + Hugging Face Transformers (NLI)
NLP	DeBERTa-v3 NLI, sentence-transformers (all-MiniLM-L6-v2), sentencepiece, langdetect, TF-IDF + SVD
OCR	Tesseract (pytesseract), Pillow
Agents / LLM	LangGraph, LangChain, Anthropic SDK (Claude Haiku 4.5), OpenAI SDK (gpt-4o-mini)
Retrieval	Wikipedia API, NewsAPI, Tavily, SerpAPI, on-disk cache
Backend	FastAPI, Uvicorn, Pydantic 2, pydantic-settings, SQLAlchemy 2, SQLite (Postgres-ready), python-jose (JWT), passlib/bcrypt, python-multipart
Frontend	Next.js 14.2 (App Router), React 18.3, axios, framer-motion, fast-xml-parser, Inter and Playfair Display fonts, plain CSS
Testing	pytest, pytest-asyncio, httpx
Visualisation	matplotlib, seaborn
Tooling	Git, Jupyter notebooks, Google Colab (planned GPU training), Swagger/OpenAPI

14. Running the System

```
# backend
pip install -r requirements.txt
copy .env.example .env         # optional API keys
uvicorn api.main:app --reload   # http://localhost:8000  (Swagger: /docs)

# frontend
cd frontend
npm install
npm run dev                    # http://localhost:3000

# tests and evaluation
pytest
python scripts/evaluate_pipeline.py
python -m paper_trail_core.registry.cli top
```

No API key is required: Wikipedia retrieval is keyless and NLI runs locally. Keys (`ANTHROPIC_API_KEY`, `NEWS_API_KEY`, etc.) enable Hindi translation and news retrieval. Tesseract and its Hindi language pack must be installed separately for image input.

14.1 End-to-end workflow example

- User pastes a screenshot of a forwarded message into /verify.
- Frontend obtains a JWT, POSTs the image to `/articles/submit-image` and receives 202 plus an article id.
- Background task: OCR → language ID → translate (if Hindi) → extract ≤5 claims → retrieve evidence → NLI per claim → stylistic score → fusion.
- Frontend polls and shows the current stage, then renders verdict, confidence, per-claim evidence and the reasoning trace.
- If the verdict is *uncertain*, the article enters the review queue, where an analyst resolves it with a human verdict and notes.

15. Conclusion

Paper Trail demonstrates that fake-news detection is more reliable when it verifies claims against evidence, calibrates its confidence and abstains when unsure. The stylistic ensemble supplies a strong prior (92% on in-distribution data) but fails on fluent fabrications. The evidence channel fixes those cases, and the fusion model combines them with a measurable precision guarantee. The system is a complete, tested, end-to-end product: multimodal input, an agentic pipeline, a secured API, a human-in-the-loop workflow and a polished web UI. Its limitations are documented, and the roadmap (multilingual Models 1-3, regional retrieval, rationale agent) is concrete.

Appendix A: Key API Schemas

```
POST /predict        {text, model_name, threshold=0.5}
  -> {model_name, display_name, category, prediction(0 fake|1 real), label,
      confidence, prob_fake, prob_real}

POST /articles/submit {text (5-50000), url?}   -> 202 {id, status}
GET  /articles/{id}   -> ArticleOut {id, url, text_preview, status, submitted_at,
                          stage, verdict: VerdictOut}
VerdictOut  {verdict, confidence, claim_verdicts[], ensemble_score, fused_score,
             reasoning_trace, needs_human_review, ai_model_version, degradations[]}
ClaimVerdictOut {claim, verdict, confidence, evidence_snippets, support,
                 entail_mass, contradict_mass, top_relevance, numeric_conflict,
                 sources, status}
```

Appendix B: Key Configuration

Variable	Default	Meaning
NLI_MODEL	MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli	Verification model
EMBED_MODEL	all-MiniLM-L6-v2	Relevance gating
RELEVANCE_MIN	0.45	Cosine floor for evidence
MAX_CLAIMS / CHECKWORTHY_MIN	5 / 0.35	Claim extraction limits
STAGE_TIMEOUT_S	90	Per-stage timeout
OCR_LANGS / OCR_CONFIDENCE_MIN	hin+eng / 0.55	OCR settings
LLM_PROVIDER / LLM_MODEL	anthropic / claude-haiku-4-5	Translation LLM
TARGET_PRECISION	0.90	Abstention-band precision target
DATABASE_URL	SQLite file	Switch to Postgres here
JWT_SECRET_KEY	dev default	Must be overridden in production

Appendix C: Repository Map

Path	Purpose
api/	FastAPI app, schemas, auth, DB, routers
paper_trail_core/	Agents, NLP, eval, registry
frontend/	Next.js web app
dataset/	Raw datasets (about 231 MB) and golden set
models/	Trained artefacts and registry.json
notebooks/	24 notebooks and about 150 figures
results/	Benchmarks and evaluation outputs
scripts/	Evaluation, fusion training, registry tools
tests/	pytest suite (123 tests)
docs/	ARCHITECTURE, CURRENT_STATE, VERIFICATION_PIPELINE
export_models.py, preprocess.py, config.py	Training entry point, data loaders, settings

Appendix D: Glossary

Term	Meaning
NLI	Natural Language Inference: classifies a hypothesis as entailed by, contradicted by, or neutral to a premise.
Stacking	An ensemble in which a meta-learner combines the predictions of base models.
Calibration	Adjusting scores so that a predicted probability matches observed frequency.
ECE	Expected Calibration Error: average gap between confidence and accuracy.
Abstention / coverage	Declining to answer when unsure; coverage is the fraction of articles that receive a verdict.
OCR	Optical Character Recognition: extracting text from images.
LangGraph	A framework for building stateful multi-step agent workflows as graphs.
Degradation	A recorded, typed note that a pipeline stage failed or fell back.